

INTRO TO SCIENTIFIC COMPUTING IN PYTHON

LECTURE 0: GETTING STARTED + BASICS OF PROGRAMMING IN PYTHON

Zan Ahmad

Department of Applied Mathematics and Statistics,
Johns Hopkins University

January 7, 2026

COURSE OVERVIEW

EN.553.285 — Introduction to Scientific Programming in Python

- ▶ See course announcement on Canvas for full details
- ▶ **2 Homeworks** (optional but strongly recommended)
 - Designed to build real problem-solving skill
 - Will be posted on Blackboard
- ▶ Office Hours: Fridays after class
- ▶ Reference text:
 - *Introduction to Scientific Computing in Python* by Robert Johansson
 - Free and openly available

This course emphasizes **thinking computationally**, not memorizing syntax.

Part I

WHY PYTHON?

PART I: WHY PYTHON FOR SCIENTIFIC PROGRAMMING?

1	Motivation	4
2	Universality and Adoption	5
3	Ease of Expression	6
4	Interpreted Language	7
5	Scientific Ecosystem	8
6	Open Source and Reproducibility	9
7	Transparency	10
8	Summary	11

WHY PYTHON FOR SCIENTIFIC PROGRAMMING?

Scientific programming is about:

- ▶ expressing mathematical ideas clearly,
- ▶ experimenting computationally,
- ▶ analyzing data and models,
- ▶ and communicating results reproducibly.

Python is designed to support all of these goals *at the same time*.

PYTHON AS A SCIENTIFIC STANDARD

Python is:

- ▶ widely used across mathematics, physics, engineering, biology, and medicine,
- ▶ the dominant language in data science and machine learning,
- ▶ heavily used in industry, research labs, and startups.

Learning Python gives you a transferable skill that applies across disciplines and careers.

Key idea: Python is not niche — it is a scientific and industrial standard.

HIGH-LEVEL VS. LOW-LEVEL LANGUAGES

Programming languages exist on a spectrum.

Low-level languages (e.g. C, C++):

- ▶ require explicit memory management,
- ▶ are fast but verbose,
- ▶ place a high cognitive burden on the programmer.

High-level languages (e.g. Python):

- ▶ abstract away low-level details,
- ▶ allow you to focus on ideas and algorithms,
- ▶ require less code to express the same concept.

Key idea: Python lets you think like a scientist, not a compiler.

INTERPRETED, NOT COMPILED

Python is an **interpreted** language.

This means:

- ▶ code is executed line-by-line,
- ▶ no explicit compilation step is required,
- ▶ rapid experimentation is possible.

For scientific work, this enables:

- ▶ interactive exploration,
- ▶ quick debugging,
- ▶ incremental model development.

Key idea: Python supports the iterative nature of the scientific method.

A RICH SCIENTIFIC ECOSYSTEM

Python's power comes from its ecosystem.

There exist thousands of high-quality, open-source packages for:

- ▶ numerical linear algebra,
- ▶ optimization,
- ▶ differential equations,
- ▶ statistics and machine learning,
- ▶ visualization and data analysis.

These packages are actively developed and widely peer-reviewed.

OPEN SOURCE AND REPRODUCIBLE SCIENCE

Python is **open-source and free**.

This matters because:

- ▶ costly proprietary software (e.g. MATLAB) introduces access barriers,
- ▶ research should be reproducible by anyone,
- ▶ tools should not depend on institutional licenses.

Key idea: Scientific results should not depend on who can afford the software.

TRANSPARENCY AND TRUST

Open-source means:

- ▶ the source code is publicly available,
- ▶ algorithms can be inspected line-by-line,
- ▶ behavior can be traced and verified.

This is essential for:

- ▶ scientific trust,
- ▶ debugging numerical methods,
- ▶ understanding assumptions in algorithms.

Official documentation is maintained at:

`www.python.org`

WHY PYTHON — SUMMARY

Python is:

- ▶ expressive and readable,
- ▶ well-suited to the scientific method,
- ▶ supported by a massive ecosystem,
- ▶ open, free, and reproducible,
- ▶ an industry and research standard.

Bottom line: Python lets you focus on *problems*, not plumbing.

Part II

PYTHON TOOLING & WORKFLOW

PART II: PYTHON TOOLING AND WORKFLOW

1	Why Tooling Matters	14
2	The Big Picture	15
3	Terminal	16
4	Conda & Environments	17
5	Python Interpreter	19
6	Jupyter	20
7	IDEs	22
8	Git & GitHub	24
9	Anaconda vs Miniconda	26
10	How It Fits Together	27

WHY TOOLING MATTERS

Programming is more than syntax.

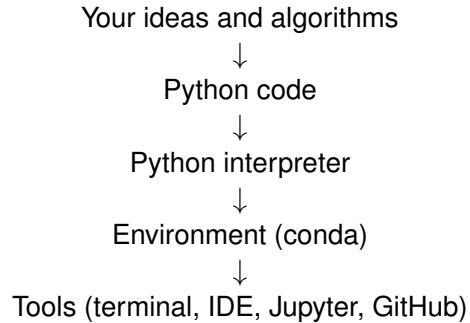
In scientific computing, you must understand:

- ▶ where your code lives,
- ▶ how it is executed,
- ▶ which Python and libraries are being used,
- ▶ how work is saved and shared.

Goal: build a clear mental model of the Python ecosystem so you can debug and work independently.

THE BIG PICTURE

Scientific programming is a layered system:



Each tool plays a distinct role. None of them replace the others.

THE COMMAND LINE (TERMINAL)

The command line is a text-based interface for interacting directly with the operating system.

You use it to:

- ▶ run Python programs,
- ▶ activate environments,
- ▶ install packages,
- ▶ launch tools such as Jupyter or VS Code.

Examples:

```
python my_script.py
conda activate sci-python
jupyter notebook
```

Key idea: The terminal launches and controls tools (it is not where you “think” through code).

CONDA AND ENVIRONMENTS

Conda is an environment and package manager. It is **not** Python itself.

A *Python environment* is a self-contained collection of:

- ▶ a specific Python interpreter,
- ▶ a specific set of installed libraries,
- ▶ specific library versions.

Why environments matter:

- ▶ different projects need different versions,
- ▶ prevents package conflicts,
- ▶ enables reproducibility.

Key idea: Conda determines which interpreter and libraries are used when your code runs.

CONDA IN PRACTICE

A typical workflow:

```
conda create -n sci-python python=3.12
conda activate sci-python
conda install numpy scipy matplotlib pandas
```

Most common mistake: installing packages in base and running code in a different environment.

THE PYTHON INTERPRETER

The Python interpreter is the program that executes Python code.

When you run:

```
python my_script.py
```

you are using the interpreter from the *active environment*.

If something fails, the first questions should be:

- ▶ Which environment is active?
- ▶ Which Python interpreter is being used?

Key idea: Many “mysterious” errors are actually “wrong environment” errors.

JUPYTER NOTEBOOKS

A Jupyter notebook is an interactive document that executes Python code in cells.

It can mix:

- ▶ code and output,
- ▶ plots,
- ▶ text explanations,
- ▶ equations (LaTeX).

Best for:

- ▶ learning and experimentation,
- ▶ data exploration,
- ▶ visualization,
- ▶ prototyping ideas.

Key idea: Jupyter is a thinking space, not a final product.

WHEN NOT TO USE JUPYTER

Jupyter is less suitable for:

- ▶ large software projects,
- ▶ long-term reproducible pipelines,
- ▶ projects that require structured modules and testing,
- ▶ heavy version control workflows.

A good habit is to prototype in a notebook, then move stable code into `.py` files.

IDES (INTEGRATED DEVELOPMENT ENVIRONMENTS)

An IDE is a full workspace for writing and maintaining code.

Examples:

- ▶ VS Code
- ▶ Spyder
- ▶ PyCharm

IDEs provide:

- ▶ code editor + auto-completion,
- ▶ debugger,
- ▶ file navigation,
- ▶ integrated terminal.

Key idea: An IDE is where code *lives* and evolves.

WHEN TO USE AN IDE

Use an IDE when you:

- ▶ write reusable `.py` files,
- ▶ organize projects across folders,
- ▶ debug non-trivial logic,
- ▶ maintain code over time.

A common workflow is:

- ▶ explore in Jupyter,
- ▶ stabilize in `.py` modules,
- ▶ run and debug in VS Code.

GIT AND GITHUB

GitHub is a platform for hosting and collaborating on code using a tool called *git*.

Distinguish:

- ▶ **git**: version control tool (tracks changes)
- ▶ **GitHub**: website that hosts git repositories

GitHub helps you:

- ▶ track changes over time,
- ▶ recover from mistakes,
- ▶ collaborate without overwriting each other's work.

Key idea: GitHub is a safety net and collaboration tool, not a programming environment.

REPOSITORIES AND A BASIC WORKFLOW

A *repository* (repo) is a folder with:

- ▶ your code files,
- ▶ a complete history of changes.

Typical workflow:

1. edit files locally (IDE / editor),
2. save a snapshot (`commit`),
3. upload to GitHub (`push`).

ANACONDA VS. MINICONDA

Both are Python distributions that include conda. They differ in what comes preinstalled.

Anaconda

- ▶ many scientific packages preinstalled,
- ▶ large install size,
- ▶ minimal setup effort,
- ▶ convenient for beginners.

Miniconda

- ▶ minimal install,
- ▶ explicit control over dependencies,
- ▶ lightweight,
- ▶ preferred for reproducibility and research workflows.

Rule of thumb: Anaconda prioritizes convenience; Miniconda prioritizes precision and control.

HOW THESE TOOLS WORK TOGETHER

A typical workflow:

1. open a terminal,
2. activate an environment,
3. launch Jupyter or VS Code,
4. write and run Python code,
5. commit and push to GitHub (when work matters).

WHEN TO USE WHAT

Task	Tool
Install packages	Conda + terminal
Explore ideas / learn	Jupyter notebook
Write structured programs	IDE + .py files
Run code	Terminal or IDE
Save / share / collaborate	Git + GitHub
Fix import errors	Check conda environment

COMMON MISCONCEPTIONS

- ▶ “Jupyter is Python” — Jupyter runs Python.
- ▶ “Conda is an IDE” — Conda manages environments.
- ▶ “VS Code replaces the terminal” — VS Code uses the terminal.
- ▶ “GitHub runs my code” — GitHub stores code; Python runs it.
- ▶ “Environments are optional” — environments are essential.

Part III

BASICS OF PROGRAMMING IN PYTHON

PART III: BASICS OF PROGRAMMING IN PYTHON

1	What is a Python Program?	32
2	Python Program Files	33
3	Modules and Imports	36
4	Variables and Types	38
5	Core Data Structures	40
6	Control Flow	43
7	Functions	47
8	Classes	50
9	Exceptions	51
10	Wrap-Up	52

PYTHON CODE: SCRIPTS VS NOTEBOOKS

In this course you will see two main ways to write Python:

1) **Python scripts:** `.py` files

- ▶ Best for reusable, organized, and version-controlled code
- ▶ Run from terminal or IDE

2) **Jupyter notebooks:** `.ipynb` files

- ▶ Best for exploration, learning, and visualization
- ▶ Mixes code + text + figures in one document

A strong workflow: prototype in notebooks, then move stable logic into `.py`.

PYTHON PROGRAM FILES (.PY)

A Python script is just a text file with Python code.

Example file: `hello.py`

```
print("Hello, world!")
```

Run it from the terminal:

```
python hello.py
```

Key idea: Running `python file.py` executes the file top-to-bottom.

SHEBANG (UNIX/MACOS)

On Unix systems, you may see a *shebang* at the top of scripts:

```
#!/usr/bin/env python
```

This tells the operating system which interpreter should run the file.

You can make a script executable (advanced / optional):

```
chmod +x myscript.py  
./myscript.py
```

In this course, it is totally fine to always run:

```
python myscript.py
```

CHARACTER ENCODING (RARE, BUT GOOD TO RECOGNIZE)

Most of the time, you do not need to think about encoding.

Sometimes you may see:

```
# -*- coding: UTF-8 -*-
```

This indicates the text encoding used by the source file.

MODULES

A **module** is code that someone else wrote that you can reuse.

Why modules matter:

- ▶ You do not want to rewrite standard tools from scratch
- ▶ Scientific computing relies on high-quality libraries
- ▶ You build on a community of tested code

Examples: `numpy`, `scipy`, `matplotlib`, `pandas`

IMPORT SYNTAX

Common import patterns:

```
import numpy
import numpy as np
from numpy import array
from numpy.linalg import norm
```

Rule of thumb:

- ▶ Use `import numpy as np` for standard scientific conventions
- ▶ Prefer explicit imports (clarity > cleverness)

VARIABLES

A variable is a name that refers to a value in memory.

You create variables by assignment:

- ▶ `x = 3`
- ▶ `name = "Zan"`
- ▶ `v = [1, 2, 3]`

Key idea: Python is dynamically typed:

- ▶ you do not declare variable types explicitly,
- ▶ types are determined at runtime.

TYPES YOU SHOULD RECOGNIZE

Fundamental built-in types:

`int, float, complex, bool, str`

Containers:

`list, tuple, dict, set`

Type inspection and casting:

`type(x)`

`isinstance(x, float)`

`float("3.14")`

`int(7.9)`

STRINGS

Strings represent text:

```
s = "hello"  
t = 'world'  
u = s + " " + t
```

Common operations:

```
len(s)  
s.upper()  
s.split()
```

Key idea: strings are immutable (operations create new strings).

LISTS AND TUPLES

Lists: mutable ordered collections

```
a = [1, 2, 3]  
a.append(4)  
a[0] = 10
```

Tuples: immutable ordered collections

```
b = (1, 2, 3)
```

Use lists when you plan to modify; tuples when you want fixed structure.

DICTIONARIES

A dictionary maps keys to values:

```
d = {"name": "Zan", "course": 285, "units": 1}
d["name"]
d.keys()
d.values()
```

Key idea: dicts are fundamental for structured data (especially in ML and APIs).

IF / ELIF / ELSE

Conditional logic:

```
x = 5
if x < 0:
    print("negative")
elif x == 0:
    print("zero")
else:
    print("positive")
```

Key idea: indentation defines blocks (there are no braces).

FOR LOOPS

Iterate over sequences:

```
for k in range(5):  
    print(k)
```

Iterate over items:

```
for item in ["a", "b", "c"]:  
    print(item)
```

LIST COMPREHENSIONS

A compact way to build lists:

```
squares = [k**2 for k in range(10)]  
evens = [k for k in range(20) if k % 2 == 0]
```

Rule of thumb: use comprehensions when they improve clarity; otherwise write a loop.

WHILE LOOPS

Repeat until a condition is false:

```
x = 1
while x < 100:
    x = 2*x
```

Warning: Always ensure the loop condition eventually becomes false.

FUNCTIONS

Functions package reusable logic:

```
def f(x):  
    return x**2 + 1
```

Why functions matter:

- ▶ reduce repetition,
- ▶ make code readable,
- ▶ isolate and test logic,
- ▶ make debugging easier.

DEFAULT AND KEYWORD ARGUMENTS

```
def power(x, p=2):  
    return x**p
```

```
power(3)          # uses default p=2  
power(3, p=3)     # keyword argument
```

Key idea: keyword arguments improve readability in scientific code.

LAMBDA (UNNAMED FUNCTIONS)

Short functions can be written anonymously:

```
g = lambda x: x**2 + 1
```

Common use: passing small functions to other functions (e.g., sorting, optimization).

CLASSES (MINIMAL VIEW)

A class bundles data + behavior.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def norm(self):
        return (self.x**2 + self.y**2)**0.5
```

In this course: understand what classes are, but we will not focus heavily on OOP.

EXCEPTIONS AND ERROR HANDLING

Errors happen. Exceptions let you handle them intentionally.

```
try:  
    x = 1 / 0  
except ZeroDivisionError:  
    print("Division by zero!")
```

Key idea: read error messages carefully; they usually tell you what happened and where.

TAKEAWAYS

After this section, you should be able to:

- ▶ write and run a simple `.py` script,
- ▶ import modules and use documentation,
- ▶ use core types and containers,
- ▶ write loops, conditionals, and comprehensions,
- ▶ write basic functions and interpret error messages.

Next: using these ideas for scientific computation (NumPy, SciPy, plotting, etc.).